

AI Document Processing System

Architecture Design Document

Automated Document Analysis, Drafting & Review using RAG + LLMs

1. Executive Summary

This document provides a comprehensive architectural design rationale for the AI Document Processing System — a multi-agent pipeline that automates document ingestion, analysis, intelligent drafting, and human-in-the-loop review. The system orchestrates Retrieval-Augmented Generation (RAG) with live web search to produce contextually accurate, up-to-date document drafts, while ensuring a human approval gate before finalizing any output.

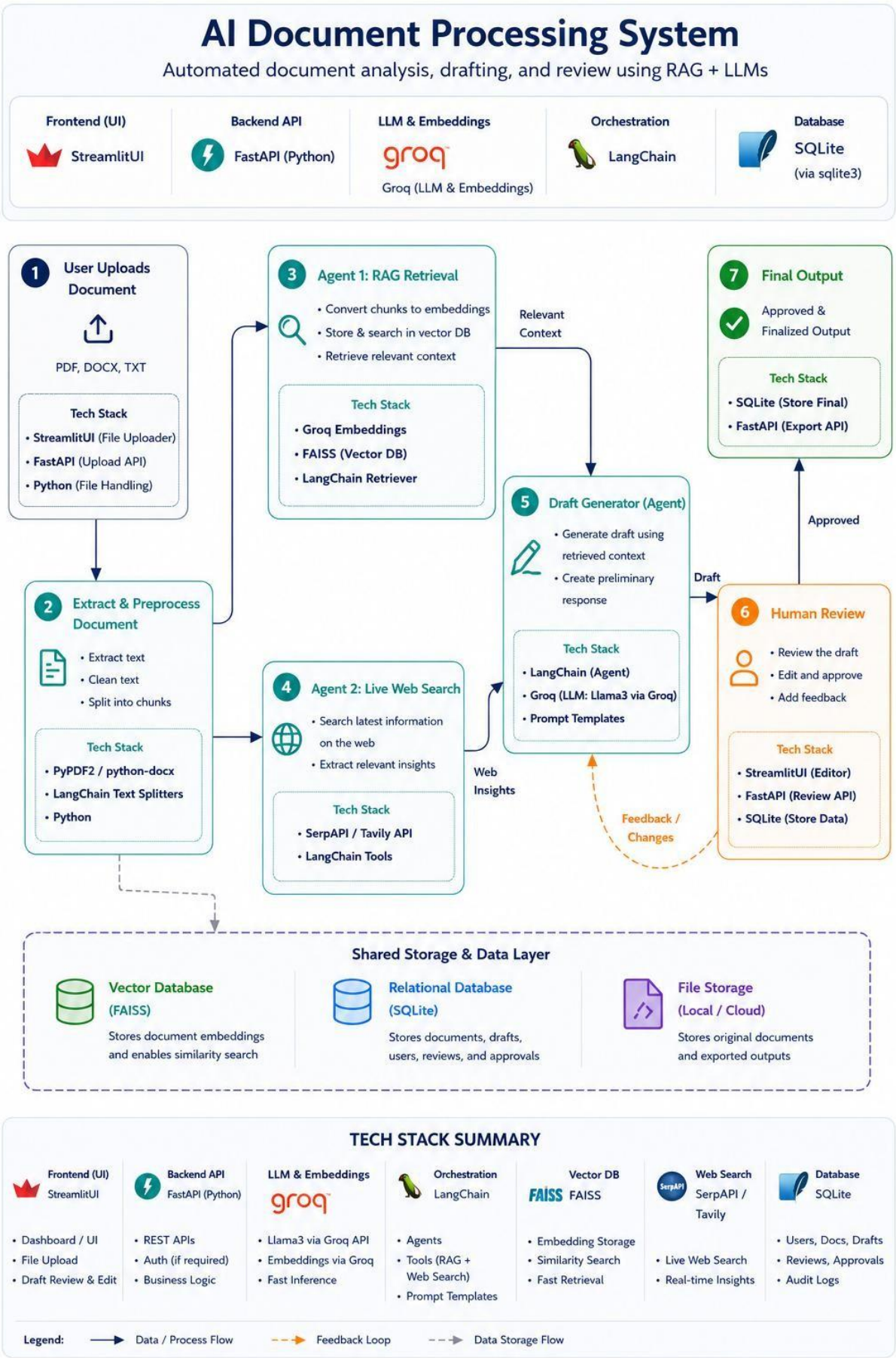
The architecture is purpose-built for accuracy, extensibility, and auditability — three non-negotiable properties for enterprise document workflows.

2. System Overview & Pipeline

The system follows a seven-stage sequential pipeline, with parallel enrichment paths converging at a unified draft generation stage:

Stage	Name	Primary Responsibility
1	User Uploads Document	Accepts PDF, DOCX, or TXT files via the Streamlit UI
2	Extract & Preprocess	Parses, cleans, and chunks document text for downstream use
3	Agent 1 — RAG Retrieval	Embeds chunks, stores in vector DB, retrieves relevant context
4	Agent 2 — Price Intelligence	Fetches capacity price from company database
5	Draft Generator (Agent)	Synthesizes retrieved context + web insights into a draft
6	Human Review	Editor reviews, annotates, and approves or requests changes
7	Final Output	Approved draft is persisted and exported via API

Stages 3 and 4 run as parallel enrichment agents feeding Stage 5. The feedback loop between Stages 5, 6, and back to 5 enables iterative refinement before final approval.



1. Technology Stack — Design Justifications

3.1 Frontend UI — Streamlit

Streamlit was chosen as the user-facing interface for its rapid prototyping capabilities and native Python integration, eliminating the need for a separate frontend engineering effort.

✓ Pros	✗ Cons
Zero-config Python-native UI — no JS required	Limited customization vs React or Vue
Built-in file upload widgets with progress indicators	Single-threaded execution can bottleneck concurrent users
Rapid iteration: UI changes reflected immediately	Not production-grade for high-traffic public applications
Ideal for internal tools and data-heavy workflows	State management is session-based and can be complex at scale
Native integration with pandas, matplotlib, LangChain	Mobile responsiveness is limited

Alternatives Considered

Alternative	Why Not Chosen
React + Next.js	Requires dedicated frontend developer; overkill for internal document tooling
Flask + Jinja2	Full templating overhead; slower iteration cycle for data apps
Dash (Plotly)	Excellent for dashboards but lacks document editing/review UI primitives

1.2 Backend API — FastAPI (Python)

FastAPI serves as the system backbone, exposing REST endpoints for file upload, review submission, and final output export. Its async-first design pairs naturally with I/O-heavy operations like embedding generation and vector DB queries.

✓ Pros	✗ Cons
Async support via asyncio — critical for nonblocking LLM calls	Younger ecosystem vs Django (fewer battletested plugins)
Automatic OpenAPI/Swagger documentation generation	No built-in ORM — requires SQLAlchemy or manual SQL
Pydantic-based request validation with minimal boilerplate	Async debugging can be complex with mixed sync/async code
Python-native — shares type system with LangChain & ML libs	Less opinionated structure can lead to inconsistency at scale
High throughput for I/O-bound operations	

Alternatives Considered

Alternative	Why Not Chosen
Django REST Framework	Heavier framework; sync-first model adds latency for LLM API calls
Flask	Lacks native async support; requires additional wiring for modern async patterns
Node.js (Express)	Context switch from Python ML stack; loses type parity with LangChain agents

3.3 LLM & Embeddings — Groq (Llama 3)

Groq provides inference for both text generation (Llama 3) and embedding creation. Its hardwareaccelerated LPU architecture delivers significantly lower latency than GPU-based alternatives, which is critical in a feedback-loop architecture where multiple LLM calls may occur per document cycle.

✓ Pros	✗ Cons
Industry-leading inference speed via LPU hardware acceleration	Groq cloud availability subject to provider reliability

Llama 3 is open-weights — auditable and finetunable	Llama 3 context window smaller than GPT-4 Turbo (128K)
Unified provider for both LLM inference and embeddings	No native function calling support (as of Llama 3.1 base)
Cost-effective relative to GPT-4 or Claude API for high-volume use	Embedding models less proven than OpenAI text-embedding-3-large
Compatible with OpenAI SDK interface — easy migration path	Vendor lock-in risk if LPU API surface changes

Alternatives Considered

Alternative	Why Not Chosen
OpenAI GPT-4o	Higher cost at scale; closed weights limit auditability
Anthropic Claude API	Excellent quality but higher cost; less LangChain embedding support

3.4 Orchestration — LangChain

LangChain serves as the orchestration layer, managing agent workflows, tool integrations (RAG retrieval + web search), prompt templating, and the multi-step reasoning chain that connects retrieval to generation.

✓ Pros	✗ Cons
Mature agent framework with pre-built RAG chains	Abstraction overhead can obscure debugging in complex chains
Native FAISS and vector DB integrations	Rapidly evolving API surface — version upgrades can be breaking
Modular tool abstractions — easy to swap retrieval or search backends	Performance overhead vs hand-rolled pipelines for simple use cases
Active community and rapid feature development	Heavy dependency footprint
LangSmith observability for tracing and debugging agent runs	

Alternatives Considered

Alternative	Why Not Chosen
-------------	----------------

LlamaIndex	Better for pure RAG/indexing; weaker multi-agent orchestration primitives
Haystack (deepset)	Strong NLP pipeline focus but smaller LLM agent ecosystem
AutoGen (Microsoft)	Multi-agent strengths but heavier setup; less RAG-native
Custom orchestration	Maximum flexibility but significant engineering cost and maintenance burden

3.5 Vector Database — FAISS

FAISS (Facebook AI Similarity Search) provides the in-process vector store for document embeddings. It enables millisecond-level similarity search without requiring a separate database server, which reduces operational complexity for the initial deployment.

✓ Pros	✗ Cons
Zero-latency in-process search — no network round-trip	In-memory — index lost on restart unless serialized to disk
Proven at scale by Meta; battle-tested on billionvector datasets	No built-in metadata filtering or hybrid search
No separate service to deploy or maintain	Scales vertically only — no horizontal distribution
LangChain native integration	No real-time index updates without full rebuild
Support for multiple index types (Flat, IVF, HNSW)	Not suitable for multi-tenant isolation without custom partitioning

Alternatives Considered

Alternative	Tradeoff vs FAISS
Pinecone	Fully managed with metadata filters; higher cost and external dependency
Weaviate	Rich hybrid search (vector + BM25); requires Docker/K8s deployment
Chroma	LangChain-native, easier dev setup; less proven at large scale
pgvector (Postgres)	Eliminates separate vector store; SQL-native but slower at large vector counts
Qdrant	High-performance with payload filtering; newer, smaller community

3.6 Relational Database — SQLite

SQLite serves as the relational store for users, documents, drafts, reviews, approvals, and audit logs. Its serverless, file-based architecture eliminates database infrastructure overhead for the initial deployment phase.

✓ Pros	✗ Cons
Zero-config serverless deployment — no DB server process	Write concurrency limited — single writer at a time Not suitable for distributed deployments or multinode scaling No native JSON indexing (available via json_each workaround) Limited data types vs PostgreSQL Migration tooling less mature than PostgreSQL/Alembic
ACID-compliant with WAL mode for concurrent reads	
Single file backup — trivial disaster recovery	
Python sqlite3 module in standard library — no dependencies	
Sufficient for thousands of documents per day without tuning	

Alternatives Considered

Alternative	When to Migrate
PostgreSQL	Recommended when concurrent write volume exceeds SQLite thresholds (>100 concurrent writers)
MySQL / MariaDB	Viable alternative to PostgreSQL; stronger replication support
MongoDB	Appropriate if document schemas become highly variable and schema-on-read is preferred

4. Data Flow Architecture

The system maintains three independent storage layers that serve distinct purposes:

Layer	Technology	Stores	Access Pattern
Vector Store	FAISS	Document chunk embeddings	Similarity search (ANN)

Relational DB	SQLite	Users, docs, drafts, reviews, audit logs	CRUD via FastAPI
File Storage	Local / Cloud	Original uploads, exported outputs	Binary read/write via Python

Data flows are categorized by type in the architecture diagram:

- Solid arrows (Data / Process Flow): sequential pipeline stages
- Dashed arrows (Data Storage Flow): reads and writes to the shared storage layer
- Orange dashed arrows (Feedback Loop): human reviewer changes cycling back to the Draft Generator

5. Human-in-the-Loop Design

The Human Review stage (Stage 6) is a deliberate architectural gate, not an optional step. This design decision reflects several critical requirements:

- Accuracy assurance: LLMs can hallucinate; human review catches factual errors before output is finalized
- Regulatory compliance: Many document workflows (legal, medical, financial) require human sign-off
- Feedback quality: Human annotations become training signal for future model fine-tuning
- Auditability: Every approval is timestamped and attributed in SQLite, creating a full audit trail

The reviewer interface (Streamlit Editor) supports inline annotation and change requests, which are fed back through the FastAPI Review API to trigger regeneration if needed. Only explicitly approved drafts proceed to Stage 7.

6. Evaluation Strategies & Logging

6.1 LLM Output Evaluation

LLM evaluation must be continuous, automated, and multi-dimensional. The following framework is applied at each draft generation cycle:

Metric	Method	Tooling	Target
Faithfulness	Does the draft only assert facts from the retrieved context?	RAGAS faithfulness scorer	> 0.85
Answer Relevancy	Is the draft responsive to the original document intent?	RAGAS answer_relevancy	> 0.80

Contextual Precision	Are retrieved chunks relevant to the question?	RAGAS context_precision	> 0.75
Hallucination Rate	Rate of claims not grounded in context or web results	LangSmith + custom NLI scorer	< 5%
Human Approval Rate	% of drafts approved without revision request	SQLite review audit log	> 70%
Revision Cycles	Avg number of feedback loops per document	SQLite review audit log	< 2.0

Evaluation Tool Justification — RAGAS

✓ Pros	✗ Cons
✓ Reference-free evaluation — no labelled ground truth needed	✗ Metrics rely on an evaluator LLM — cost per evaluation run
✓ LangChain native integration	✗ Scores can be gamed by verbose but low-information drafts
✓ Covers faithfulness, relevancy, precision in one framework	✗ Does not natively evaluate multi-document synthesis quality
✓ Open-source; customisable scorers	✗ Requires careful threshold calibration per use case
✓ Produces numeric scores suitable for CI/CD gating	

Alternatives — LLM Evaluation Frameworks

Alternative	Tradeoff
DeepEval	Strong assertion-based testing; less RAG-native than RAGAS
TruLens	Excellent LLM tracing; primarily for OpenAI models
Promptfoo	Best for prompt A/B testing; less suited for RAG pipeline metrics
Human eval only	Ground truth but expensive, slow, and unscalable
BERTScore	Semantic similarity; does not assess factual faithfulness

6.2 RAG Retrieval Evaluation

Retrieval quality is evaluated independently from generation quality to isolate failure modes between the vector search and the LLM drafting layers:

Metric	Description	Target
Context Recall	Fraction of ground-truth facts present in retrieved chunks	> 0.80

MRR@K	Mean Reciprocal Rank of the first relevant chunk at K=5	> 0.75
Hit Rate@K	% of queries where at least one relevant chunk is in top K	> 0.85
Chunk Overlap Rate	% of retrieved chunks with high cosine overlap (redundancy)	< 20%
Embedding Drift	Cosine distance shift of embeddings after model updates	< 0.05

6.3 System Performance Metrics

Metric	Measurement Point	SLO Target
End-to-end latency (upload → draft)	Stage 1 → Stage 5	< 15 seconds P95
Embedding generation throughput	Stage 3 (RAG Agent)	> 100 chunks/sec
Vector search latency	FAISS similarity search	< 50ms P99
Web search latency	Stage 4 (SerpAPI/Tavily)	< 2s P95
API response time (FastAPI)	All REST endpoints	< 200ms P95
Draft approval time (human)	Stage 6 (human)	< 30 min median
System availability	Full pipeline uptime	> 99.5% monthly

6.4 Logging Architecture

Structured logging is applied at every pipeline stage to enable debugging, compliance auditing, and performance regression detection:

Log Category	Fields Captured	Storage	Retention
Request logs	timestamp, user_id, document_id, file_type, size_bytes	SQLite + file	90 days
Pipeline stage logs	stage_id, duration_ms, tokens_in, tokens_out, status	SQLite	90 days
LLM call logs	model, prompt_hash, completion_tokens, latency_ms, cost_usd	LangSmith	30 days
Retrieval logs	query_embedding_hash, top_k_scores, chunk_ids_returned	SQLite	30 days
Review audit logs	reviewer_id, action (approve/reject/edit), timestamp, diff_hash	SQLite	7 years

Error logs	stage, exception_type, stack_trace, document_id	File + alert	180 days
------------	---	--------------	----------

Logging Design Justification

✓ Pros	✗ Cons
✓ Structured JSON logs are machine-queryable — no regex parsing	✗ High-cardinality logs grow storage quickly — need retention policies
✓ Per-stage duration logging isolates bottlenecks precisely	✗ LangSmith external dependency adds cost and data egress
✓ Prompt hash logging avoids storing PII while enabling deduplication	✗ Stack trace logging may expose internal architecture details
✓ Review audit logs support regulatory compliance (GDPR, SOC 2)	✗ Log volume spikes during peak processing can overwhelm local storage
✓ Cost_usd field enables per-document LLM cost attribution	

6.5 Observability Stack

The observability stack extends structured logging with distributed tracing, metric dashboards, and alerting:

Layer	Chosen Tool	Justification
LLM Tracing	LangSmith	Native LangChain integration; step-level chain tracing
Metrics & Dashboards	Prometheus + Grafana	Industry-standard; open-source; FastAPI exporter available
Log Aggregation	Loki (or ELK)	Grafana-native (Loki) for unified dashboard; ELK for richer search
Alerting	Grafana Alerting	Unified alert rules co-located with dashboards
Error Tracking	Sentry	Python native; captures stack traces with context
Uptime Monitoring	UptimeRobot / Freshping	Lightweight HTTP endpoint monitoring; zero-infra

Alternatives — Observability

Alternative	Tradeoff vs Chosen Stack
Datadog	Excellent APM but expensive at scale; vendor lock-in
New Relic	Full-stack observability; cost scales with data volume
OpenTelemetry (OTEL)	Vendor-neutral; requires backend (Jaeger/Tempo) — more setup

AWS CloudWatch	Zero-infra on AWS; limited cross-cloud portability
No observability	Unacceptable for production — impossible to debug or SLA-manage

7. Scalability Considerations & Evolution Path

7.1 Current Architecture (Phase 1)

- Single-server deployment with in-process FAISS and SQLite
- Suitable for up to ~500 documents/day with sub-second retrieval • File storage on local disk with optional cloud sync

7.2 Phase 2 Scaling Triggers

Trigger	Recommended Upgrade
Concurrent writes > 50/min	Migrate SQLite to PostgreSQL
Vector index > 10M chunks	Migrate FAISS to Weaviate or Pinecone
Multi-user concurrent editing	Add WebSocket layer; separate Streamlit into dedicated React frontend
Multi-tenant isolation required	Introduce namespace partitioning in vector store; row-level security in DB
LLM latency becomes bottleneck	Implement async batch processing queue (Celery + Redis)

8. Security Considerations

- Authentication: FastAPI OAuth2 / JWT middleware (noted as optional in current design)
- File validation: MIME type and size validation on upload to prevent malicious file injection
- API key management: Groq, SerpAPI, and Tavily keys stored in environment variables, never in code
- Data isolation: Each document processing session is namespaced by user/session ID
- Audit logging: All review actions and approvals written to SQLite with timestamps and user attribution
- LLM prompt injection: Input sanitization applied before chunks are inserted into prompt templates

9. Key Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
LLM hallucination in drafts	Medium	High	Mandatory human review gate; RAG grounding reduces hallucination rate
Groq API outage	Low	High	Implement fallback to Ollama (local) or OpenAI as secondary provider
FAISS index loss on restart	Medium	Medium	Serialize index to disk after every ingestion; reload on startup
SQLite write contention	Medium	Low	WAL mode enabled; plan PostgreSQL migration at scale threshold
Sensitive document exposure	Low	High	Enforce auth on all API endpoints; encrypt file storage at rest

10. Conclusion

The AI Document Processing System architecture represents a pragmatic balance between development velocity, operational simplicity, and production readiness. The technology choices — Streamlit, FastAPI, Groq/Llama 3, LangChain, FAISS, and SQLite — are individually well-justified and collectively coherent: they share a Python-native ecosystem, minimize external service dependencies, and enable a single developer or small team to build, test, and deploy the full pipeline rapidly.

